

Lista de exercícios 3

Projeto e análise de algoritmos

***Lembre-se que é fundamental ler e compreender as seções indicadas nos slides para realizar a prova.**

Programação dinâmica

1. Considere uma modificação do problema do corte da haste no qual, além de um preço p_i para cada haste, cada corte incorre em um custo fixo c . A receita associada à solução, agora, é a soma dos preços das peças menos os custos da execução dos cortes. Dê um algoritmo de programação dinâmica para resolver esse problema modificado.
2. Modifique CORTA-HASTE e CORTA-HASTE-MEMOIZADO de modo que seus laços for subam para apenas $\lfloor n/2 \rfloor$, em vez de subir para n . Que outras mudanças de procedimentos precisariam ser feitas? Como os tempos de execução seriam afetados?
3. Modifique CORTA-HASTE-MEMOIZADO para retornar não somente o valor, mas também a solução propriamente dita.
4. Os números de Fibonacci são definidos pela recorrência:

$$F_i = \begin{cases} 0 & \text{se } i = 0, \\ 1 & \text{se } i = 1, \\ F_{i-1} + F_{i-2} & \text{se } i \geq 2 \end{cases}$$

Forneça um algoritmo de programação dinâmica de tempo $O(n)$ para calcular o n -ésimo número de Fibonacci.

5. Determine uma parentização ótima de um produto de cadeias de matrizes cuja sequência de dimensões é $\langle 5, 10, 3, 12, 5, 50, 6 \rangle$.
6. Forneça um algoritmo recursivo MULTIPLICA-CADEIA-MATRIZES(A, s, i, j) que realmente execute a multiplicação ótima de cadeias de matrizes, dadas a sequência de matrizes $\langle A_1, A_2, \dots, A_n \rangle$, a tabela s calculada por ORDENA-CADEIA-DE-MATRIZES e os índices i e j . (A chamada inicial seria MULTIPLICA-CADEIA-MATRIZES($A, s, 1, n$).) Suponha que a chamada MULTIPLICA-MATRIZ-RETANGULAR(A, B) retorne o produto das matrizes A e B .
7. Qual modo é mais eficiente para determinar o número ótimo de multiplicações em um problema de multiplicação de cadeias de matrizes: enumerar todos os modos de parentizar o produto e calcular o número de multiplicações para cada um ou executar CADEIA-DE-MATRIZES-RECURSIVO? Justifique sua resposta.
8. Desenhe a árvore de recursão para o procedimento MERGE-SORT em um vetor de 16 elementos. Explique por que a memoização não aumenta a velocidade de um bom algoritmo de divisão e conquista como MERGE-SORT.
9. Considere uma variante contrária do problema da multiplicação de cadeias de matrizes na qual a meta é parentizar a sequência de matrizes de modo a maximizar, em vez de minimizar, o número de multiplicações escalares. Esse problema exhibe subestrutura ótima?
10. Determine uma LCS de $\langle 1, 0, 0, 1, 0, 1, 0, 1 \rangle$ e $\langle 0, 1, 0, 1, 1, 0, 1, 1, 0 \rangle$.
11. Dê o pseudocódigo para reconstruir uma LCS partindo da tabela c concluída e das sequências originais $X = \langle x_1, x_2, \dots, x_m \rangle$ e $Y = \langle y_1, y_2, \dots, y_n \rangle$ em tempo $O(m + n)$, sem usar a tabela b .

12. Dê uma versão memoizada de COMPRIMENTO-LCS que seja executada no tempo $O(mn)$.

Algoritmos gulosos

1. (Problema das atividades) Suponha que, em vez de sempre selecionarmos a primeira atividade a terminar, selecionemos a última atividade a começar que seja compatível com todas as atividades selecionadas anteriormente. Descreva como essa abordagem é um algoritmo guloso e prove que ela produz uma solução ótima.
2. Forneça uma solução de programação dinâmica para o problema binário da mochila que seja executado no tempo $O(nW)$, em que n é o número de itens e W é o peso máximo de itens que o ladrão pode colocar em sua mochila.
3. (Huffman) Prove que uma árvore binária que não é estritamente binária não pode corresponder a um código de prefixo ótimo.
4. Qual é o código de Huffman ótimo para o conjunto de frequências a seguir, baseado nos oito primeiros números de Fibonacci?
a:1 b:1 c:2 d:3 e:5 f:8 g:13 h:21
Generalize sua resposta para determinar o código ótimo quando as frequências são os primeiros n números de Fibonacci?

Análise amortizada

1. Se o conjunto de operações de pilha incluísse uma operação MULTIPUSH, que introduz k itens na pilha, o limite $O(1)$ para o custo amortizado de operações de pilha continuaria válido?
2. Use a análise agregada para determinar o custo amortizado por operação para uma sequência de n operações em uma estrutura de dados na qual a i -ésima operação custa i , se i é uma potência exata de 2, e 1 em caso contrário.

Problemas NP-Completo

1. O algoritmo de programação dinâmica para o problema da mochila 0-1 apresentado na aula sobre algoritmos gulosos é um algoritmo em tempo polinomial? Explique sua resposta.
2. Suponha que os algoritmos ALG e ALG' transformam cadeias de caracteres em outras cadeias de caracteres. O algoritmo ALG consome $O(n^2)$ unidades de tempo e o algoritmo ALG' consome $O(n^4)$ unidades de tempo, onde n é o número de caracteres da cadeia de entrada. Considere agora o algoritmo ALGALG' que consiste na composição de ALG e ALG', com ALG' recebendo como entrada a saída de ALG. Qual o consumo de tempo de ALGALG'?
3. Seja ALG um algoritmo que faz um número constante de chamadas a um algoritmo ALG'. Suponha que se o consumo de tempo de ALG' é constante então o consumo de tempo de ALG é polinomial.
 - a. Mostre que se o consumo de tempo de ALG' é polinomial então o consumo de tempo de ALG é polinomial.
 - b. Mostre que se o consumo de tempo de ALG' é polinomial e ALG faz um número polinomial de chamadas a ALG', então é possível que o consumo de tempo de ALG seja exponencial.
4. Mostre que se $\Pi_1 \leq_P \Pi_2$ e $\Pi_2 \leq_P \Pi_3$, então $\Pi_1 \leq_P \Pi_3$.

Respostas de alguns exercícios (em inglês)

Programação dinâmica

1. Considere uma modificação do problema do corte da haste no qual, além de um preço p_i para cada haste, cada corte incorre em um custo fixo c . A receita associada à solução, agora, é a soma dos preços das peças menos os custos da execução dos cortes. Dê um algoritmo de programação dinâmica para resolver esse problema modificado.

MODIFIED-CUT-ROD(p, n, c)

 let $r[0:n]$ be a new array

$r[0] = 0$

for $j = 1$ **to** n

$q = p[j]$

for $i = 1$ **to** $j - 1$

$q = \max(q, p[i] + r[j - i] - c)$

$r[j] = q$

return $r[n]$

The major modification required is in the body of the inner **for** loop, which now reads $q = \max(q, p[i] + r[j - i] - c)$. This change reflects the fixed cost of making the cut, which is deducted from the revenue. We also have to handle the case in which we make no cuts (when i equals j); the total revenue in this case is simply $p[j]$. Thus, we modify the inner **for** loop to run from i to $j - 1$ instead of to j . The assignment $q = p[j]$ takes care of the case of no cuts. If we did not make these modifications, then even in the case of no cuts, we would be deducting c from the total revenue.

2. Modifique CORTA-HASTE e CORTA-HASTE-MEMOIZADO de modo que seus laços **for** subam para apenas $\lfloor n/2 \rfloor$, em vez de subir para n . Que outras mudanças de procedimentos precisariam ser feitas? Como os tempos de execução seriam afetados?

CUT-ROD(p, n)

```

if  $n == 0$ 
    return 0
 $q = p[n]$ 
for  $i = 1$  to  $\lfloor n/2 \rfloor$ 
     $q = \max \{q, p[i] + \text{CUT-ROD}(p, n - i)\}$ 
return  $q$ 

```

MEMOIZED-CUT-ROD-AUX(p, n, r)

```

if  $r[n] \geq 0$ 
    return  $r[n]$ 
 $q = p[n]$ 
for  $i = 1$  to  $\lfloor n/2 \rfloor$ 
     $q = \max \{q, p[i] + \text{MEMOIZED-CUT-ROD-AUX}(p, n - i, r)\}$ 
 $r[n] = q$ 
return  $q$ 

```

Note that in addition to changing the loop bounds, for both procedures, instead of initializing $q = -\infty$, initialize $q = p[n]$. In MEMOIZED-CUT-ROD-AUX, we can also remove the case for $n = 0$.

The memoized code still takes $\Theta(n^2)$ time. The running time for the recursive CUT-ROD procedure reduces to $\Theta(2^{n/2})$.

3. Modifique CORTA-HASTE-MEMOIZADO para retornar não somente o valor, mas também a solução propriamente dita.

MEMOIZED-CUT-ROD(p, n)

```

let  $r[0:n]$  and  $s[0:n]$  be new arrays
for  $i = 0$  to  $n$ 
     $r[i] = -\infty$ 
 $(val, s) = \text{MEMOIZED-CUT-ROD-AUX}(p, n, r, s)$ 
print "The optimal value is "  $val$  " and the cuts are at "
 $j = n$ 
while  $j > 0$ 
    print  $s[j]$ 
     $j = j - s[j]$ 

```

MEMOIZED-CUT-ROD-AUX(p, n, r, s)

```

if  $r[n] \geq 0$ 
    return  $r[n]$ 
if  $n == 0$ 
     $q = 0$ 
else  $q = -\infty$ 
    for  $i = 1$  to  $n$ 
         $(val, s) = \text{MEMOIZED-CUT-ROD-AUX}(p, n - i, r, s)$ 
        if  $q < p[i] + val$ 
             $q = p[i] + val$ 
             $s[n] = i$ 
 $r[n] = q$ 
return  $(q, s)$ 

```

PRINT-CUT-ROD-SOLUTION constructs the actual lengths where a cut should happen. Array entry $s[i]$ contains the value j indicating that an optimal cut for a rod of length i is j inches. The next cut is given by $s[i - j]$, and so on.

4. Os números de Fibonacci são definidos pela recorrência:

$$F_i = \begin{cases} 0 & \text{se } i = 0, \\ 1 & \text{se } i = 1, \\ F_{i-1} + F_{i-2} & \text{se } i \geq 2 \end{cases}$$

Forneça um algoritmo de programação dinâmica de tempo $O(n)$ para calcular o n -ésimo número de Fibonacci.

FIBONACCI(n)

```

    let  $fib[0:n]$  be a new array
     $fib[0] = fib[1] = 1$ 
    for  $i = 2$  to  $n$ 
         $fib[i] = fib[i - 1] + fib[i - 2]$ 
    return  $fib[n]$ 

```

FIBONACCI directly implements the recurrence relation of the Fibonacci sequence. Each number in the sequence is the sum of the two previous numbers in the sequence. The running time is clearly $O(n)$.

The subproblem graph consists of $n + 1$ vertices, $v_0, v_1, \dots, v_n$. For $i = 2, 3, \dots, n$, vertex v_i has two leaving edges: to vertex v_{i-1} and to vertex v_{i-2} . No edges leave vertices v_0 or v_1 . Thus, the subproblem graph has $2n - 2$ edges.

5. Determine uma parentização ótima de um produto de cadeias de matrizes cuja sequência de dimensões é $\langle 5, 10, 3, 12, 5, 50, 6 \rangle$.

The m and s tables:

m	j					
	1	2	3	4	5	6
1	0	150	330	405	1655	2010
2		0	360	330	2430	1950
i 3			0	180	930	1770
4				0	3000	1860
5					0	1500
6						0

s	j				
	2	3	4	5	6
1	1	2	2	4	2
2		2	2	2	2
i 3			3	4	4
4				4	4
5					5

The optimal parenthesization printed is $((A_1 A_2)((A_3 A_4)(A_5 A_6)))$.

6. Forneça um algoritmo recursivo `MULTIPLICA-CADEIA-MATRIZES(A, s, i, j)` que realmente execute a multiplicação ótima de cadeias de matrizes, dadas a sequência de matrizes $\langle A_1, A_2, \dots, A_n \rangle$, a tabela s calculada por `ORDENA-CADEIA-DE-MATRIZES` e os índices i e j . (A chamada inicial seria `MULTIPLICA-CADEIA-MATRIZES(A, s, 1, n)`.) Suponha que a chamada `MULTIPLICA-MATRIZ-RETANGULAR(A, B)` retorne o produto das matrizes A e B .

MATRIX-CHAIN-MULTIPLY(A, s, i, j)

if $j > i$

$X = \text{MATRIX-CHAIN-MULTIPLY}(A, s, i, s[i, j])$

$Y = \text{MATRIX-CHAIN-MULTIPLY}(A, s, s[i, j] + 1, j)$

return `MATRIX-MULTIPLY`(X, Y)

else return A_i

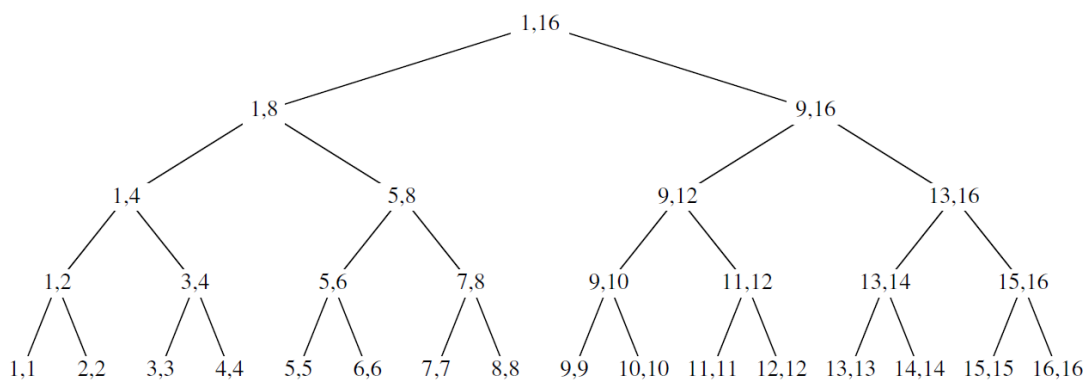
7. Qual modo é mais eficiente para determinar o número ótimo de multiplicações em um problema de multiplicação de cadeias de matrizes: enumerar todos os modos de parentizar o produto e calcular o número de multiplicações para cada um ou executar `CADEIA-DE-MATRIZES-RECURSIVO`? Justifique sua resposta.

Running RECURSIVE-MATRIX-CHAIN is asymptotically more efficient than enumerating all the ways of parenthesizing the product and computing the number of multiplications for each.

Consider the treatment of subproblems by the two approaches.

- For each possible place to split the matrix chain, the enumeration approach finds all ways to parenthesize the left half, finds all ways to parenthesize the right half, and looks at all possible combinations of the left half with the right half. The amount of work to look at each combination of left- and right-half subproblem results is thus the product of the number of ways to do the left half and the number of ways to do the right half.
- For each possible place to split the matrix chain, RECURSIVE-MATRIX-CHAIN finds the best way to parenthesize the left half, finds the best way to parenthesize the right half, and combines just those two results. Thus the amount of work to combine the left- and right-half subproblem results is $O(1)$.

8. Desenhe a árvore de recursão para o procedimento MERGE-SORT em um vetor de 16 elementos. Explique por que a memoização não aumenta a velocidade de um bom algoritmo de divisão e conquista como MERGE-SORT.



Looking at the recursion tree for MERGE-SORT, we see that there are no overlapping subproblems: every problem solved is unique. Therefore, memoization would not decrease the amount of work done and would fail to speed up MERGE-SORT. In general, good divide-and-conquer algorithms consider each subproblem only once, so that memoization does not speed them up.

9. Considere uma variante contrária do problema da multiplicação de cadeias de matrizes na qual a meta é parentizar a sequência de matrizes de modo a maximizar, em vez de minimizar, o número de multiplicações escalares. Esse problema exhibe subestrutura ótima?

Yes, this problem exhibits optimal substructure.

Since the subproblems are independent, a cut-and-paste argument works. Suppose that we have a parenthesization P of a sequence of matrices $A_1, \dots, A_j$ that maximizes the number of scalar multiplications. Split the sequence between any two matrices A_k and A_{k+1} . If this problem did not exhibit optimal substructure, there could be a parenthesization $A_1, \dots, A_k$ with even more scalar multiplications. We could then replace the parenthesization of $A_1, \dots, A_k$ in P to achieve an even greater number of scalar multiplications, contradicting our assumption.

10. Determine uma LCS de $\langle 1, 0, 0, 1, 0, 1, 0, 1 \rangle$ e $\langle 0, 1, 0, 1, 1, 0, 1, 1, 0 \rangle$.

The LCS-LENGTH procedure finds the LCS $\langle 1, 0, 0, 1, 1, 0 \rangle$. The sequences $\langle 1, 0, 0, 1, 0, 1, 0, 1 \rangle$ and $\langle 0, 1, 0, 1, 1, 0, 1, 1, 0 \rangle$ have four other LCSs: $\langle 0, 1, 0, 1, 0, 1 \rangle$, $\langle 1, 0, 1, 0, 1, 0 \rangle$, $\langle 1, 0, 1, 0, 1, 1 \rangle$, and $\langle 1, 0, 1, 1, 0, 1 \rangle$.

11. Dê o pseudocódigo para reconstruir uma LCS partindo da tabela c concluída e das sequências originais $X = \langle x_1, x_2, \dots, x_m \rangle$ e $Y = \langle y_1, y_2, \dots, y_n \rangle$ em tempo $O(m + n)$, sem usar a tabela b .

```

RECONSTRUCT-LCS( $c, X, Y, i, j$ )
  if  $i == 0$  or  $j == 0$ 
    return
  if  $x_i == y_j$ 
    RECONSTRUCT-LCS( $c, X, Y, i - 1, j - 1$ )
    print  $x_i$ 
  elseif  $c[i, j] == c[i - 1, j]$ 
    RECONSTRUCT-LCS( $c, X, Y, i - 1, j$ )
  else RECONSTRUCT-LCS( $c, X, Y, i, j - 1$ )

```

This procedure emulates the PRINT-LCS procedure. When x_i and y_i are the same, then the LCS must have been extended to add this symbol, so that the procedure prints the LCS of X_{i-1} and Y_{j-1} and then prints x_i . Otherwise, $c[i, j]$ must equal either $c[i - 1, j]$ or $c[i, j - 1]$ (or both). If $c[i, j]$ equals $c[i - 1, j]$, then the LCS-LENGTH procedure would have put “↑” into $b[i, j]$, and so RECONSTRUCT-LCS recurses with $i - 1$ and j . Otherwise, LCS-LENGTH would have put “←” into $b[i, j]$, and so RECONSTRUCT-LCS recurses with i and $j - 1$.

12. Dê uma versão memoizada de COMPRIMENTO-LCS que seja executada no tempo $O(mn)$.

```

MEMOIZED-LCS-LENGTH( $X, Y, m, n$ )
  let  $c[1 : m, 1 : n]$  be a new table
  for  $i = 0$  to  $m$ 
    for  $j = 0$  to  $n$ 
       $c[i, j] = \text{NIL}$ 
  return LOOKUP-LCS( $c, m, n$ )

LOOKUP-LCS( $c, i, j$ )
  if  $c[i, j] \neq \text{NIL}$ 
    return  $c[i, j]$ 
  if  $x_i == y_j$ 
     $c[i, j] = \text{LOOKUP-LCS}(i - 1, j - 1) + 1$ 
  else  $c[i, j] = \max \{ \text{LOOKUP-LCS}(i - 1, j), \text{LOOKUP-LCS}(i, j - 1) \}$ 
  return  $c[i, j]$ 

```

Algoritmos gulosos

1. Suponha que, em vez de sempre selecionarmos a primeira atividade a terminar, selecionemos a última atividade a começar que seja compatível com todas as atividades selecionadas anteriormente. Descreva como essa abordagem é um algoritmo guloso e prove que ela produz uma solução ótima.

The proposed approach—selecting the last activity to start that is compatible with all previously selected activities—is really the greedy algorithm but starting from the end rather than the beginning.

Another way to look at it is as follows. We are given a set $S = \{a_1, a_2, \dots, a_n\}$ of activities, where $a_i = [s_i, f_i)$, and we propose to find an optimal solution by selecting the last activity to start that is compatible with all previously selected activities. Instead, let us create a set $S' = \{a'_1, a'_2, \dots, a'_n\}$, where $a'_i = [f_i, s_i)$. That is, a'_i is a_i in reverse. Clearly, a subset of $\{a_{i_1}, a_{i_2}, \dots, a_{i_k}\} \subseteq S$ is mutually compatible if and only if the corresponding subset $\{a'_{i_1}, a'_{i_2}, \dots, a'_{i_k}\} \subseteq S'$ is also mutually compatible. Thus, an optimal solution for S maps directly to an optimal solution for S' and vice versa.

The proposed approach of selecting the last activity to start that is compatible with all previously selected activities, when run on S , gives the same answer as the greedy algorithm from the text—selecting the first activity to finish that is compatible with all previously selected activities—when run on S' . The solution that the proposed approach finds for S corresponds to the solution that the text's greedy algorithm finds for S' , and so it is optimal.

2. Forneça uma solução de programação dinâmica para o problema binário da mochila que seja executado no tempo $O(nW)$, em que n é o número de itens e W é o peso máximo de itens que o ladrão pode colocar em sua mochila.

The solution is based on the optimal-substructure observation in the text: Let i be the highest-numbered item in an optimal solution S for W pounds and items $1, \dots, n$. Then $S' = S - \{i\}$ must be an optimal solution for $W - w_i$ pounds and items $1, \dots, i - 1$, and the value of the solution S is v_i plus the value of the subproblem solution S' .

We can express this relationship in the following formula: Define $c[i, w]$ to be the value of the solution for items $1, \dots, i$ and maximum weight w . Then

$$c[i, w] = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0, \\ c[i - 1, w] & \text{if } w_i > w, \\ \max \{v_i + c[i - 1, w - w_i], c[i - 1, w]\} & \text{if } i > 0 \text{ and } w \geq w_i. \end{cases}$$

The last case says that the value of a solution for i items either includes item i , in which case it is v_i plus a subproblem solution for $i - 1$ items and the weight excluding w_i , or doesn't include item i , in which case it is a subproblem solution for $i - 1$ items and the same weight. That is, if the thief picks item i , then v_i value is added, and the thief can choose from items $1, \dots, i - 1$ up to the weight limit $w - w_i$, gaining $c[i - 1, w - w_i]$ additional value. On the other hand, if the thief decides not to take item i , then choices remain from items $1, \dots, i - 1$ up to the weight limit w , giving $c[i - 1, w]$ value. The better of these two choices should be made.

The algorithm takes as inputs the maximum weight W , the number n of items, and the two sequences $v = \langle v_1, v_2, \dots, v_n \rangle$ and $w = \langle w_1, w_2, \dots, w_n \rangle$. It stores the $c[i, j]$ values in a table $c[0:n, 0:W]$ whose entries are computed in row-major order. (That is, the first row of c is filled in from left to right, then the second row, and so on.) At the end of the computation, $c[n, W]$ contains the maximum value the thief can take.

DYNAMIC-0-1-KNAPSACK(v, w, n, W)

```

let  $c[0:n, 0:W]$  be a new array
for  $w = 0$  to  $W$ 
     $c[0, w] = 0$ 
for  $i = 1$  to  $n$ 
     $c[i, 0] = 0$ 
    for  $w = 1$  to  $W$ 
        if  $w_i \leq w$  and  $v_i + c[i - 1, w - w_i] > c[i - 1, w]$ 
             $c[i, w] = v_i + c[i - 1, w - w_i]$ 
        else  $c[i, w] = c[i - 1, w]$ 
```

We can use the c table to deduce the set of items to take by starting at $c[n, W]$ and tracing where the optimal values came from. If $c[i, w] = c[i - 1, w]$, then item i is not part of the solution, and we continue tracing with $c[i - 1, w]$. Otherwise item i is part of the solution, and we continue tracing with $c[i - 1, w - w_i]$.

The above algorithm takes $\Theta(nW)$ time total:

- $\Theta(nW)$ to fill in the c table: $(n+1) \cdot (W+1)$ entries, each requiring $\Theta(1)$ time to compute.
- $O(n)$ time to trace the solution (since it starts in row n of the table and moves up one row at each step).

3. Prove que uma árvore binária que não é estritamente binária não pode corresponder a um código de prefixo ótimo.

Let T be a nonfull binary tree. Then T contains an internal node u with only one child, v . Replace edge (u, v) by edges from u to the child or children of v , obtaining a tree T' . Tree T' represents a better coding than T because all the characters in the subtree rooted at v have one bit removed from their codewords. Thus, a nonfull binary tree does not correspond to a prefix-free code.

4. Qual é o código de Huffman ótimo para o conjunto de frequências a seguir, baseado nos oito primeiros números de Fibonacci?

a:1 b:1 c:2 d:3 e:5 f:8 g:13 h:21

Generalize sua resposta para determinar o código ótimo quando as frequências são os primeiros n números de Fibonacci?

An optimal Huffman code for the first 8 Fibonacci numbers:

frequency	character	codeword
1	a	0000000
1	b	0000001
2	c	000001
3	d	00001
5	e	0001
8	f	001
13	g	01
21	h	1

In general, after merging i nodes, the tree is a rightgoing spine of internal nodes, with each internal node having a leaf as its left child and the deepest internal node having a and b as its children. The frequency in the root after merging i nodes is $F_{i+2} - 1$, which can be shown by induction. Since $F_{i+2} - 1 = F_i + F_{i+1} - 1$ and the character with frequency F_i has already been merged after merging i nodes, the next merge takes the root with frequency $F_{i+2} - 1$ and the character with frequency F_{i+1} .

To generalize the code for the first n Fibonacci numbers, the code for the character with frequency F_i is 0^{n-1} if $i = 1$ and $0^{n-i}1$ if $i \geq 2$, where 0^j represents a string of j 0s.

Análise amortizada

1. Se o conjunto de operações de pilha incluísse uma operação MULTIPUSH, que introduz k itens na pilha, o limite $O(1)$ para o custo amortizado de operações de pilha continuaria válido?

With a MULTIPUSH operation, the amortized cost of stack operations would no longer be $O(1)$. The cost of a single MULTIPUSH that pushes k items onto the stack is $\Theta(k)$.

2. Use a análise agregada para determinar o custo amortizado por operação para uma sequência de n operações em uma estrutura de dados na qual a i -ésima operação custa i , se i é uma potência exata de 2, e 1 em caso contrário.

Let c_i = cost of i th operation.

$$c_i = \begin{cases} i & \text{if } i \text{ is an exact power of } 2, \\ 1 & \text{otherwise.} \end{cases}$$

Operation	Cost
1	1
2	2
3	1
4	4
5	1
6	1
7	1
8	8
9	1
10	1
$\vdots$	$\vdots$

n operations cost

$$\sum_{i=1}^n c_i \leq n + \sum_{j=0}^{\lg n} 2^j = n + (2n - 1) < 3n.$$

(Note: Ignoring floor in upper bound of $\sum 2^j$.)

$$\text{Average cost of operation} = \frac{\text{Total cost}}{\# \text{ operations}} < 3.$$

By aggregate analysis, the amortized cost per operation = $O(1)$.